Golden Model and Quantisation Report

FPGA-Based MAC-Array Accelerator for Real-Time ECG Arrhythmia Classification

Phase 1 and Phase 2.1 · S Harish Adithya, Rajalakshmi Engineering College · 9 October 2026 · rev 1.0

1. Summary

This report covers the software half of the project: the trained neural network that every later hardware block must reproduce (the *golden model*), and the conversion of that network from 32-bit floating point to the 16-bit fixed-point numbers the FPGA uses (*quantisation*). It merges and updates the earlier Phase 1 and Phase 2.1 reports, includes the activation-range correction that was found later, and adds checks that were run for this report directly on the saved weight files and on the 16 real test beats that the hardware is verified against.

Table 1. The project's numeric design at a glance.

Item	Result
Task	Classify one ECG heartbeat as Normal or Abnormal (A, V, L and R beats pooled as Abnormal). MIT-BIH Arrhythmia Database, 360 Hz, lead MLII, 260 samples per beat.
Network	conv1 (1→8, kernel 5) → ReLU → max-pool 2 → conv2 (8→16, kernel 5) → ReLU → max-pool 2 → fc1 (992→32) → ReLU → fc2 (32→2) → compare the two scores.
Size	32,546 parameters (32,488 weights + 58 biases). 121,408 multiplications per beat.
Evaluation	Patient-independent: 38 records for training, 10 unseen records for testing (24,526 test beats).
Float32 result	Accuracy 0.99, Abnormal precision 0.98, recall 0.93, F1 0.95 (two decimals, as printed by scikit-learn; threshold 0.5).
Fixed-point result	Accuracy 98.51 % (24,160 of 24,526 beats), Abnormal precision 0.98, recall 0.93, with real 16-bit saturation at every layer.
Number formats	Input Q4.12; conv1 weights and bias Q2.14; all other weights and biases Q1.15; layer outputs Q6.10 / Q6.10 / Q7.9 / Q6.10; 48-bit accumulator.
Saturation	7 events in 784,832 fc1 outputs on the whole test set; none in conv1, conv2 or fc2.

Main finding of the quantisation work. The fixed-point model reproduces the float model's Abnormal-class precision and recall to two decimals, and on 16 real beats its scores differ from the float model by at most 0.0070 (about 7 steps of 1/1024) with identical decisions. The activation formats were corrected once, before any hardware was built on them: the first plan (one Q2.14 format everywhere) would have overflowed by 6.6× to 13.7× on real data.

2. Data and labels

2.1 Source and beat extraction

The data is the MIT-BIH Arrhythmia Database, read with the `wfdb` Python package: 48 half-hour two-lead recordings at 360 Hz, each with expert annotations marking the position and clinical type of every heartbeat. For each annotated beat, a window of 260 samples (130 on either side of the annotated R-peak) is cut from the MLII lead. Only the symbols N (normal), A (atrial premature), V (ventricular premature), L and R (left and right bundle branch block) are kept; rhythm-change markers are discarded, and a beat whose window would run past either end of its record is skipped.

2.2 Two classes instead of five

A, V, L and R are merged into one **Abnormal** class, so the task is Normal versus Abnormal. Three reasons, decided at the start:

- Atrial premature beats (A) are too rare to learn a reliable separate decision boundary.
- A single binary output needs fewer output bits and one comparator in hardware.
- "Flag this beat for review" is a realistic screening use.

2.3 Patient-independent split

Beats from one patient are strongly alike, so a random split of beats lets the network recognise the patient rather than the arrhythmia. The split is therefore made by **record**: all beats of a test record stay out of training.

Table 2. Class counts in the patient-independent split.

Set	Records	Beats	Normal	L	R	V	A	Abnormal total
Train	38	75,497	54,531	8,071	7,256	3,202	2,437	20,966 (27.8 %)
Test	10 (unseen)	24,526	20,490	0	0	3,927	109	4,036 (16.5 %)

Test records: 101, 106, 116, 119, 200, 208, 215, 221, 223, 234. The other 38 records are used for training. The recorded class counts show that the test records contain only V and A beats as Abnormal; bundle-branch-block beats (L, R) are learned from the training records but are not present in the test set.

Normalisation. Each beat is standardised as (x − mean) / std using the training-set statistics only: mean = −0.32793 mV, std = 0.45691 mV. Raw sample ranges are [−4.45, 3.64] mV in the training set and [−5.12, 5.115] mV in the test set. These constants were not recorded when the model was trained; they were recomputed by re-running the deterministic extraction, and the beat counts and class counts were confirmed identical to the original run before use.

3. The golden model

The network is a small 1-D convolutional network, kept small on purpose so that it can be built from one shared multiply-accumulate unit and fits in on-chip memory: two convolution blocks followed by two fully connected layers.

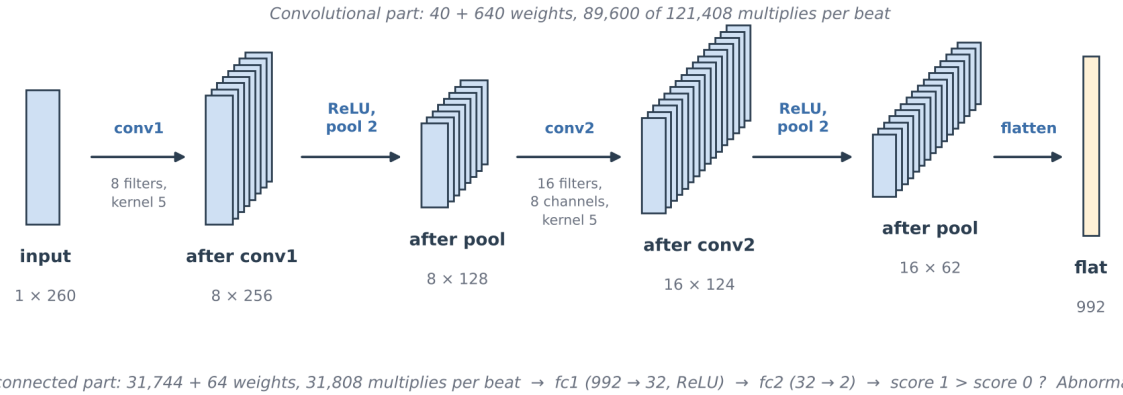


Figure 1. Network structure. Each stack of sheets is one tensor; the number of sheets is the number of channels. Sizes are channels × samples.

Table 3. Layer sizes and the cost of one heartbeat.

Layer	Operation	Output shape	Weights	Biases	Multiplications per beat
conv1	Conv1D, kernel 5, 1→8 channels	8 × 256 (pooled: 8 × 128)	40	8	8 × 256 × 5 = 10,240
conv2	Conv1D, kernel 5, 8→16 channels	16 × 124 (pooled: 16 × 62)	640	16	16 × 124 × (8 × 5) = 79,360
fc1	Linear 992 → 32 (flatten of 16 × 62)	32	31,744	32	992 × 32 = 31,744
fc2	Linear 32 → 2	2	64	2	32 × 2 = 64
Total			32,488	58	121,408

Weights versus multiplications. A convolution weight is reused at every sliding-window position, so the multiplication count (121,408) is larger than the weight count (32,488); a fully connected weight is used once. The weight count sets the memory size, the multiplication count sets the time. The two numbers were confused in the first version of the Phase 1 report; the 121,408 figure was confirmed by counting every multiplication in the fixed-point forward pass.

Training set-up. PyTorch, float32; cross-entropy loss with class weights 0.5554 (Normal) and 1.4446 (Abnormal), computed as inverse class frequency scaled to sum to 2; Adam, learning rate 0.001; *full-batch* training, so one epoch is exactly one parameter update. No dropout, no augmentation, no learning-rate schedule.

4. Training and evaluation

4.1 First attempt: beat-level split (rejected)

The first version used five records (100, 106, 119, 200, 223; 11,457 beats: N 9,058, A 135, V 2,264) pooled and split at random 80/20. It scored 98.39 % (Abnormal recall 0.94, precision 0.98). Because beats of the same patient were on both sides, this number measures memorisation of patients as well as arrhythmia recognition, so it was discarded as the project's quality figure and replaced by the patient-independent split of Section 2.3.

4.2 Patient-independent model and the training-length investigation

The first patient-independent run (40 epochs) scored 81.11 % accuracy with Abnormal precision 0.46 and recall 0.85, and its precision swung from 0.37 to 0.97 as the threshold moved from 0.3 to 0.8. The loss was still falling at a nearly constant rate, which points to an under-trained model rather than a bad architecture. Training was repeated from fresh weights for 80 and then 120 epochs (Figure 2, Table 4).

A jump from 0.46 to 0.96 precision from longer training alone is surprising, so before accepting it the following were checked: the 38/10 patient split was unchanged and non-overlapping; the test set size and class counts (20,490 / 4,036) were identical; the model was freshly initialised, not a continuation; and train accuracy (95.75 %) was compared with test accuracy (97.92 %). Test accuracy above train accuracy is the opposite of overfitting. The gap is explained by class mix: the training set is 27.8 % Abnormal and the test set 16.5 %, and Abnormal is the harder class. Computing overall accuracy from the per-class recalls reproduces both figures to within one percentage point. The improvement was accepted as genuine, and a further run to 120 epochs gave the final model.

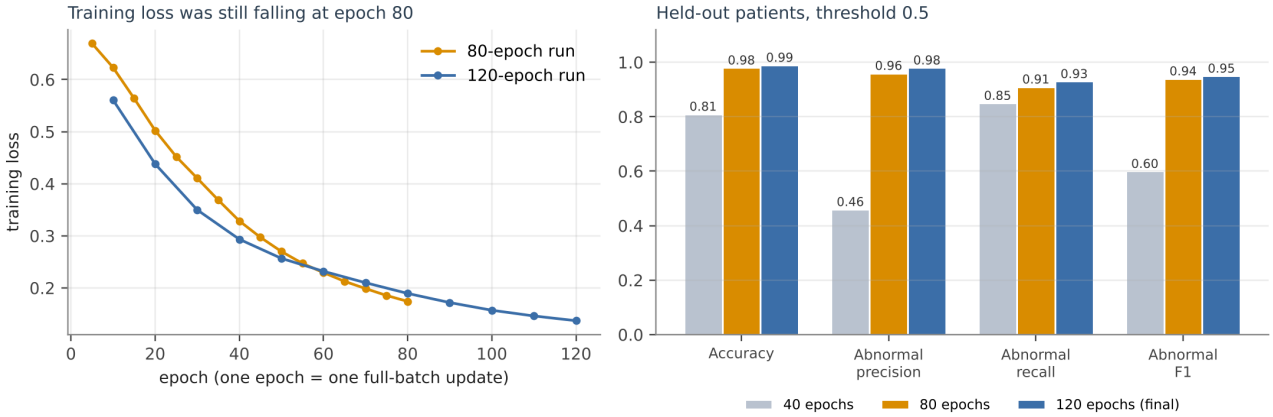


Figure 2. Left: training loss logged during the 80- and 120-epoch runs (separate runs from fresh initial weights). Right: test results on the 10 held-out records at threshold 0.5. The 40-epoch values are those recorded at the time.

Table 4. Effect of training length on the held-out records.

Metric	40 epochs	80 epochs	120 epochs (final)
Train accuracy	not recorded	95.75 %	96.77 %
Test accuracy	81.11 %	97.92 %	0.99 (two decimals)
Abnormal precision	0.46	0.96	0.98
Abnormal recall	0.85	0.91	0.93
Abnormal F1	0.60	0.94	0.95
Confusion matrix [[TN, FP],[FN, TP]]	n/a	[[20345, 145], [365, 3671]]	n/a

4.3 Decision threshold

Between thresholds 0.4 and 0.7 the F1-score stays at 0.95, so the model is not sensitive to the exact cut-off. Threshold 0.5 gives the highest accuracy (0.99) and is the choice. It has a hardware advantage: the softmax probability of Abnormal exceeds 0.5 exactly when score 1 exceeds score 0, so the hardware needs only one signed comparison of the two raw scores and no softmax. A tie is defined as Normal in both the Python model and the hardware.

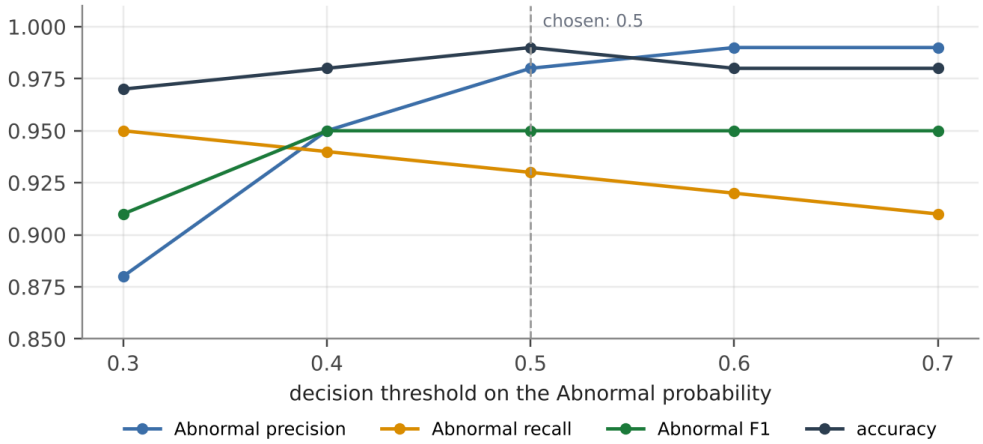


Figure 3. Threshold sweep of the final (120-epoch) model on the test records. The curves are flat from 0.4 to 0.7.

Honest limits of these numbers. The number of epochs and the threshold were chosen by looking at test-set results; there was no separate validation set. The reported accuracy is therefore slightly optimistic. The comparison with published results in the earlier Phase 1 report is only indicative, because published figures use different splits, class mixes and preprocessing; this model is a compact two-class design on a split of the project's own choosing, not a claim to exceed the state of the art.

5. Quantisation

5.1 Fixed-point numbers and the Q notation

FPGA logic handles integers cheaply and floating point expensively. A fixed-point number is an integer with an implied binary point: in format **Qa.b** a 16-bit signed integer *n* stands for the value n / 2^b, where *a* counts the sign bit plus the integer bits and *b* the fractional bits (a + b = 16). More fractional bits give finer steps; more integer bits give a larger range. Every value in the network is stored in 16 bits; the five formats in use differ only in where the point sits (Figure 4).

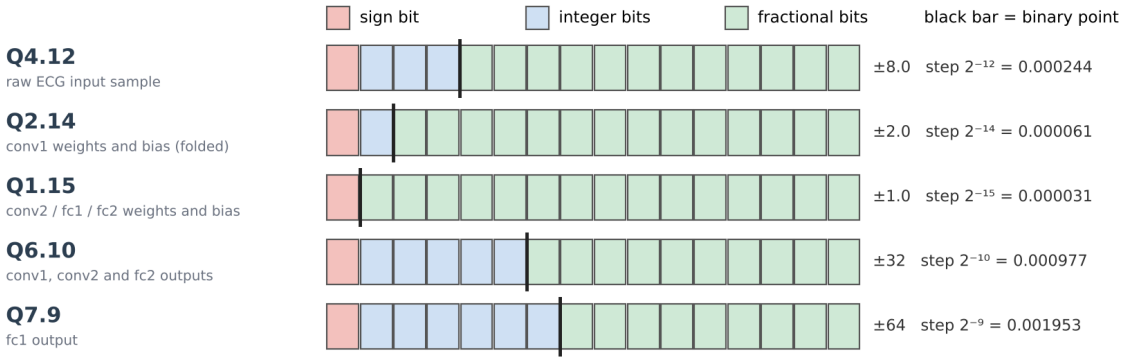


Figure 4. The five 16-bit formats used in the design, with the largest magnitude each can hold.

5.2 Weights: range analysis and format choice

Before choosing a format the actual range of each layer's trained weights and biases was measured (Table 5, Figure 5). The largest magnitude of any parameter is 0.5215, so a format with a range of ±1 and 15 fractional bits (Q1.15, step 0.000031) holds every un-folded parameter with the finest resolution available.

Table 5. Range of the trained float32 parameters, measured on ecg_cnn_weights_v2_final.pkl.

Layer	Weight min	Weight max	Bias min	Bias max
conv1 (before folding)	−0.4892	0.5215	−0.2441	0.4202
conv2	−0.2633	0.2432	−0.2093	0.2222
fc1	−0.1806	0.1786	−0.0321	0.0574
fc2	−0.2426	0.2671	0.1376	0.1575

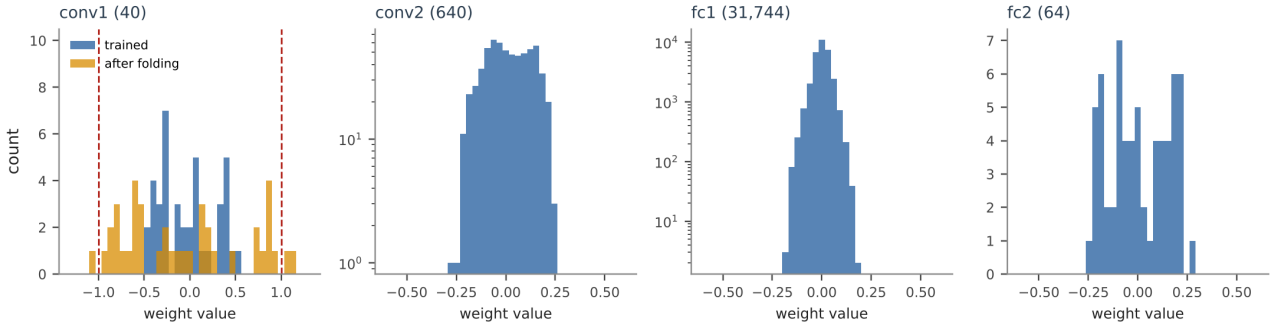


Figure 5. Distribution of the trained weights per layer (log count axis for conv2 and fc1). For conv1, the orange bars show the weights after the normalisation folding of Section 5.3; the dashed lines mark the ±1 limit of Q1.15.

Check run for this report. For conv2, fc1 and fc2 the stored 16-bit integers were compared with round(weight × 2^{15}) computed from the float file: all 32,448 weights and 50 biases agree exactly, and the largest conversion error is 0.0000153, i.e. half a step, the theoretical minimum for rounding.

5.3 Folding the input normalisation into conv1

The model standardises each input as x_{norm} = (x − mean) / std. Doing this in hardware would need a subtraction and a division per sample. Because conv1 is linear, the standardisation can instead be absorbed into conv1's own parameters:

$$y = \Sigma\, w \cdot (x - \text{mean})/\text{std} + b \;=\; \Sigma\, (w/\text{std}) \cdot x + [\, b - (\text{mean}/\text{std}) \cdot \Sigma\, w \,]$$

so with $w' = w / \text{std}$ and $b' = b - (\text{mean}/\text{std}) \cdot \Sigma w$ (the sum over the filter's five weights), conv1 can be fed the **raw** sample directly. The change is algebraically exact, costs nothing at run time and removes the division.

Side effect. Since std = 0.4569 is below 1, dividing by it enlarges the weights. The largest folded conv1 weight is 1.1414 and the largest folded bias is 1.1268; three of the eight filters (3, 4 and 6) have a weight above 1 in magnitude, and filter 6's bias is above 1 as well. These values do not fit in Q1.15 (maximum 0.99997) and would wrap silently. conv1 therefore uses Q2.14 (range ±2, step 0.000061). The other layers are unaffected by folding and keep Q1.15.

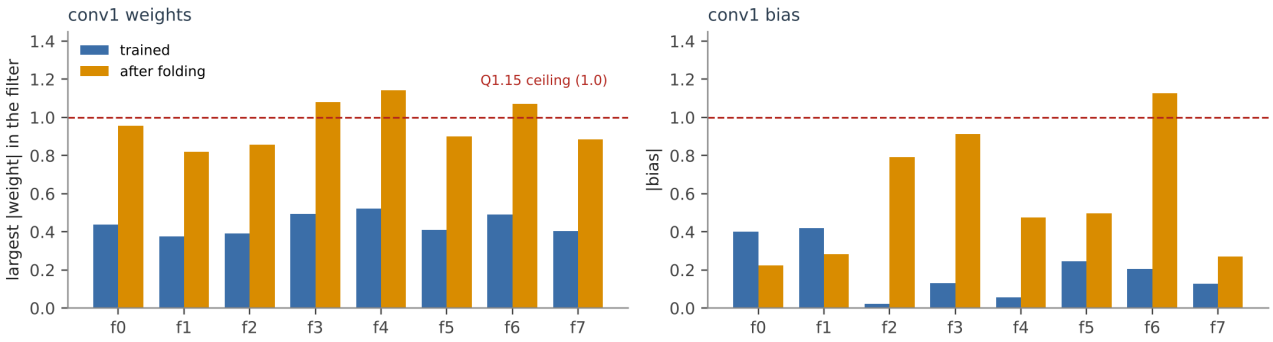


Figure 6. conv1 parameters per filter before and after folding. Folding raises the largest weight from 0.5215 to 1.1414, past the Q1.15 limit.

Check run for this report. The 40 stored conv1 weights and 8 biases equal round(folded value × 2^{14}) computed from the float file and the stored mean and std; the largest conversion error is 0.0000293 (limit: half a step, 0.0000305).

5.4 The input format

Raw samples are in millivolts with a measured range of ±5.12. Q4.12 (range ±8, step 0.000244 mV) covers this with headroom for patient-to-patient variation and no clipping was needed on the test set. Because normalisation is folded into conv1, the hardware input is the raw sample converted with round(x × 4096).

5.5 Arithmetic across a layer

Multiplying a value with p fractional bits by a weight with q fractional bits gives a product with $p + q$ fractional bits, and the products of one output are added in a wide **accumulator**. Then, for every output value:

- the bias is shifted **left** until it has the accumulator's number of fractional bits, and added;
- the sum is shifted **right** (arithmetic shift: rounds toward minus infinity, no rounding term, exactly as Python's >>) to the layer's output format;
- the result is **saturated** to the 16-bit range (clamped, never wrapped), then ReLU is applied (conv1, conv2, fc1; not fc2, whose raw scores may be negative);
- after conv1 and conv2, max-pool 2 keeps the larger of each neighbouring pair.

The accumulator is 48 bits wide. The worst case is fc1: 992 products of at most 2^{30} each, below 2^{40}, which needs 41 signed bits. The layer-controller test in the RTL feeds all 992 terms at −32768 and obtains 1,065,151,889,408 = 992 × 2^{30} without overflow.

5.6 Correction: activation ranges measured on real data

Phase 2.1 first rescaled every layer's output to the same Q2.14 format (range ±2), chosen because it matched the weight format. This was an assumption; the Python simulation used unbounded integers and so never tested it. When the 16-bit limit was added to the hardware plan, the simulation was instrumented to record the largest value at every layer over the complete 24,526-beat test set (Table 6, Figure 7).

Table 6. Measured activation ranges and the corrected formats.

Layer	Largest value measured	Times the Q2.14 limit (±2)	New format	Format limit	Margin
conv1	13.26	6.6×	Q6.10	±32	2.4×
conv2	16.44	8.2×	Q6.10	±32	1.9×
fc1	27.32	13.7×	Q7.9	±64	2.3×
fc2	14.13 (range −12.09 to 14.13)	7.1×	Q6.10	±32	2.3×

With Q2.14 and saturation, most activations in every layer would have clamped to the same ceiling and accuracy would have collapsed; with wrap-around the result would have been silently wrong. The fix needed no extra hardware: the words stay 16 bits wide and only the position of the binary point moves. The fc1 output gets one more integer bit (Q7.9) because it has the largest peak. The 98.50 % result of Phase 2.1 stayed valid, since it never applied a 16-bit limit; the mistake was only in the plan for storing values in 16 bits. It was found before any RTL was built on it.

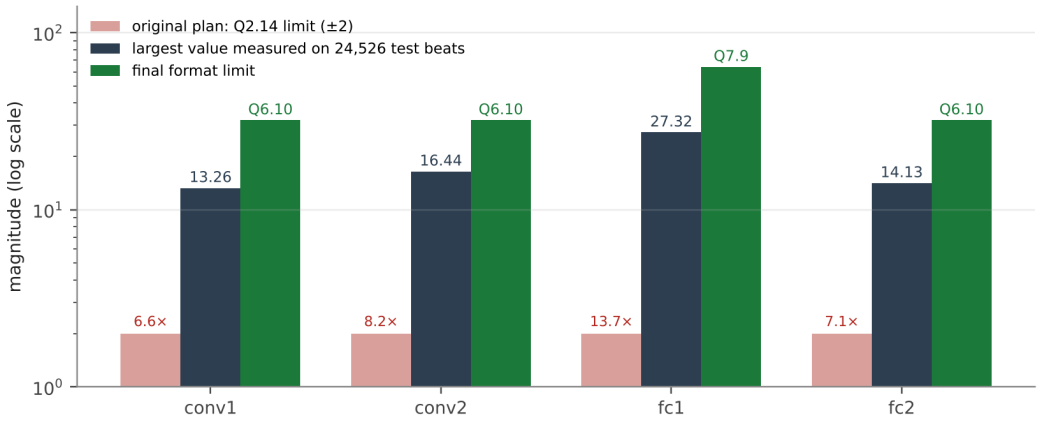


Figure 7. Largest activation per layer on the 24,526 test beats, compared with the original Q2.14 limit and the final formats (log scale).

5.7 The final numeric scheme

Table 7. Shift amounts per layer. Right-shift = accumulator fractional bits − output fractional bits.

Layer	Input format	Weight format	Accumulator fractional bits	Bias left-shift	Output right-shift	Output format	ReLU
conv1	Q4.12	Q2.14	26	12	16	Q6.10	yes
conv2	Q6.10	Q1.15	25	10	15	Q6.10	yes
fc1	Q6.10	Q1.15	25	10	16	Q7.9	yes
fc2	Q7.9	Q1.15	24	9	14	Q6.10	no

6. Validation of the fixed-point model

6.1 Whole test set

A pure-integer forward pass (NumPy int64 accumulators, explicit shifts, 16-bit saturation after every layer) was run on all 24,526 test beats, working on raw samples with no normalisation step.

Table 8. Whole-test-set comparison.

Model	Accuracy	Abnormal precision	Abnormal recall	Abnormal F1
Float32 (Phase 1, 120 epochs)	0.99 (two decimals)	0.98	0.93	0.95
Fixed-point, unbounded integers, Q2.14 everywhere (Phase 2.1)	98.50 % (24,159 correct)	0.98	0.93	0.95
Fixed-point, 16-bit saturation, per-layer formats (final)	98.51 % (24,160 correct)	0.98	0.93	0.95

The Phase 2.1 confusion matrix was [[20396, 94], [273, 3763]] (specificity 0.995). The final model classifies one more beat correctly. Because the float model's accuracy was only printed to two decimals (0.99), the exact cost of quantisation in accuracy cannot be stated more precisely than "at most about half a percentage point"; the Abnormal-class precision, recall and F1 are unchanged to two decimals. A formatting error in any bit-width or shift would have produced a collapse, not this small difference.

6.2 Saturation events

Table 9. Clamping events with real 16-bit saturation.

Layer	Values produced (24,526 beats)	Saturation events
conv1	50,229,248 (8 × 256 per beat)	0
conv2	48,659,584 (16 × 124 per beat)	0
fc1	784,832 (32 per beat)	7 (≈ 0.0009 %)
fc2	49,052 (2 per beat)	0

The seven events show that saturation is a working safety net rather than an unused one: the corrected formats have real but small margin, and rare outliers still reach it. fc1 is the layer with the most terms per output (992), so small rounding differences have the most chance to add up to an unusually large value.

6.3 Float32 versus fixed-point on 16 real beats

The hardware is verified against 16 real test beats chosen on purpose: typical Normal and Abnormal beats, beats with the largest activations, beats where fc1 saturates, a false positive, a false negative, and the two correct calls closest to the decision boundary. For this report the float32 network was run on the same 16 beats and compared with the fixed-point scores (Table 10, Figure 8).

Table 10. The 16 key beats: float32 model and fixed-point model.

#	Test beat	True	Float32 scores (0 / 1)	Fixed-point scores (0 / 1)	Float decision	Fixed decision
0	19138	Normal	1.4938 / -1.3476	1.4893 / -1.3457	Normal	Normal
1	12443	Abnormal	-6.8299 / 8.2845	-6.8301 / 8.2842	Abnormal	Abnormal
2	5748	Normal	0.5881 / 3.6447	0.5811 / 3.6475	Abnormal	Abnormal
3	5681	Normal	0.6737 / 2.2149	0.6689 / 2.2168	Abnormal	Abnormal
4	5208	Normal	-3.3612 / 0.0392	-3.3633 / 0.0381	Abnormal	Abnormal
5	5747	Normal	-0.8881 / 6.0759	-0.8926 / 6.0771	Abnormal	Abnormal
6	5730	Normal	-3.4250 / 0.2017	-3.4277 / 0.2012	Abnormal	Abnormal
7	12676	Normal	-1.6716 / 4.1773	-1.6709 / 4.1729	Abnormal	Abnormal
8	19716	Abnormal	1.0371 / -0.6432	1.0352 / -0.6436	Normal	Normal
9	14558	Normal	0.6163 / 0.6124	0.6133 / 0.6133	Normal	Normal
10	13914	Normal	0.5907 / 0.5887	0.5889 / 0.5879	Normal	Normal
11	7223	Normal	2.3157 / -1.9908	2.3125 / -1.9893	Normal	Normal
12	3520	Abnormal	-1.7543 / 3.2159	-1.7520 / 3.2109	Abnormal	Abnormal
13	5413	Normal	2.0428 / -1.3318	2.0430 / -1.3330	Normal	Normal
14	3887	Abnormal	-1.8075 / 2.1490	-1.8076 / 2.1475	Abnormal	Abnormal
15	13487	Normal	1.8402 / -0.9133	1.8369 / -0.9131	Normal	Normal

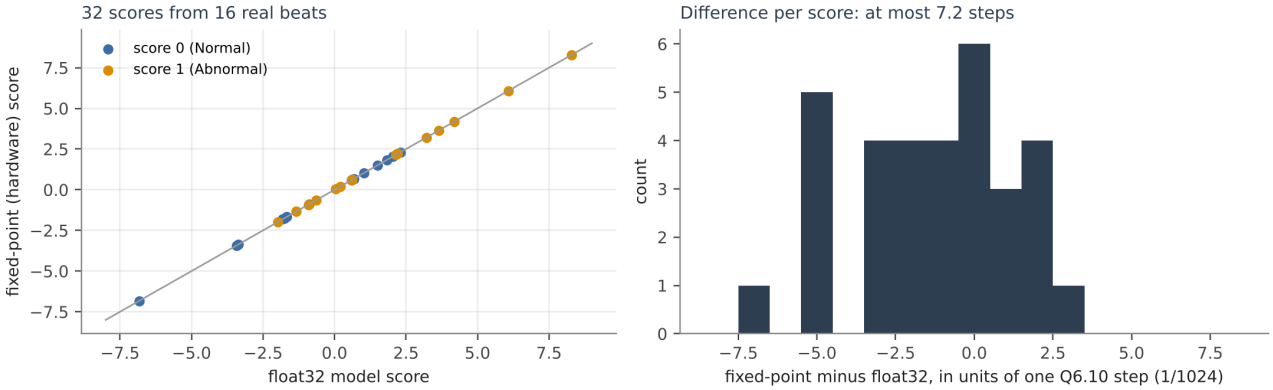


Figure 8. Left: the 32 scores from the 16 beats, float32 against fixed-point (the diagonal is perfect agreement). Right: the difference per score in units of one Q6.10 step.

The largest difference of any score is 0.0070 (7.2 steps of 1/1024) and the mean absolute difference is 0.0021; all 16 decisions agree, including the two beats closest to the boundary (beat 9: float margin 0.0039, fixed-point an exact tie, which is defined as Normal; beat 10: margins 0.0020 and 0.0010). Beats 7 and 8 are wrong in *both* models, which is the intended behaviour: the hardware must reproduce the model's mistakes, not correct them. These 16 beats are an illustration, not a statistic; the whole-test-set result above is the accuracy figure.

6.4 What the network sees

Figure 9 follows one Normal and one Abnormal beat through the fixed-point network, using the exact integer values the hardware must produce (converted back to real numbers for display). In the figure, different conv1 and conv2 filters respond to the sharp R-peak of the Normal beat and to the wide, slow shape of the Abnormal beat, and fc1 compresses these into 32 numbers from which fc2 forms the two scores.

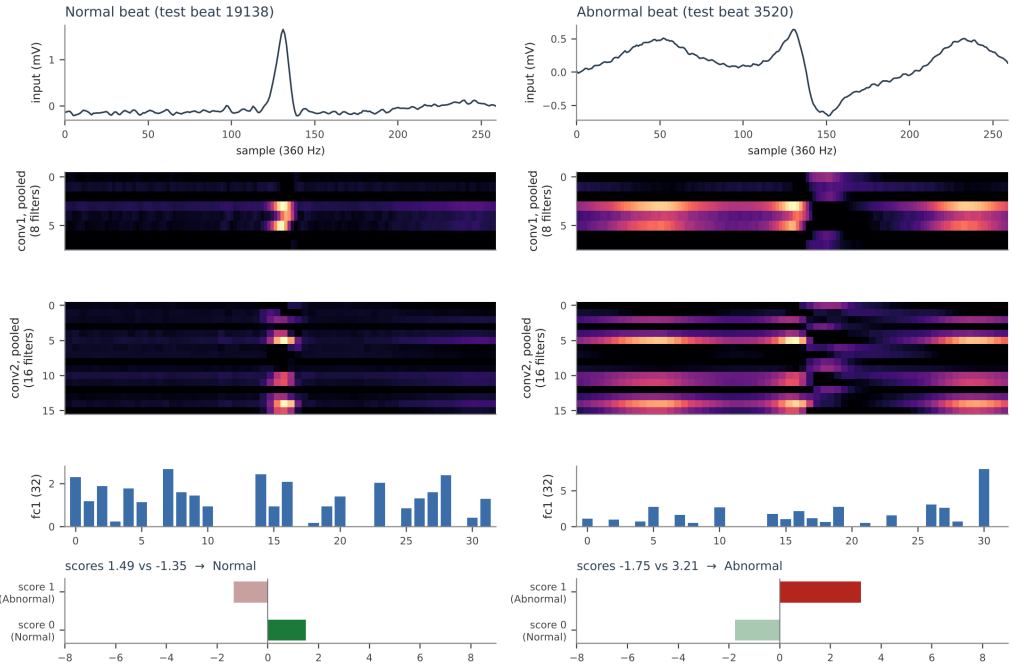


Figure 9. A Normal beat (left) and an Abnormal beat (right) through the fixed-point network. Rows: input; conv1 after ReLU and pooling (8 filters × 128 positions); conv2 after ReLU and pooling (16 × 62); fc1 outputs (32); the two class scores. Brighter colour means a larger activation.

7. What is handed to the hardware

Table 11. Files that carry the golden model into the RTL work.

Artifact	Content and use
ecg_cnn_weights_v2_final.pkl	Trained float32 weights of the 120-epoch model.
ecg_cnn_weights_fixedpoint.pkl	int16 weights and biases (conv1 folded, Q2.14; others Q1.15), the input format, and the normalisation constants kept for reference.
export_rom_files.py → weights.hex, biases.hex	32,488 weights and 58 biases as 4-digit hex, in the order the address generator walks them (filter → channel → tap). A built-in self-check replays all 121,408 multiplications' weight addresses against the pickle. Layer start offsets: weights conv1 0, conv2 40, fc1 680, fc2 32,424; biases 0, 8, 24, 56.
fixed_point_forward_v2.py	The reference integer model: the numbers every RTL result is compared with.
make_answer_key.py → answer_key/	Expected values of every stage (conv1/conv2 before and after pooling, fc1, scores, decision) for the 16 key beats. The script checks that its fast vectorised copy equals a loop-for-loop copy of the reference, equals the scores saved from the original session, and reproduces 98.51 % and the saturation counts on the full test set.

Two conventions must be matched exactly by any hardware: right shifts are arithmetic with no rounding term, and a tie between the two scores is Normal.

8. Limitations and open points

- **Selection on the test set.** Epochs and threshold were chosen with test-set results visible; there was no validation set (Section 4.3).
- **Own split, two classes.** The task is Normal versus Abnormal on a 38/10 split of the project's own choosing, not a standard published split, so cross-paper accuracy comparisons are indicative only.
- **No L or R beats in the test set.** Bundle-branch-block beats are in training but not in the test records, and no per-class result is reported for A, V, L, R.
- **Float accuracy only to two decimals.** The exact accuracy loss from quantisation is bounded (about 0.5 percentage points or less) but not measured to more digits.
- **Activation ranges were measured on the test set only.** Margins of 1.9× to 2.4× were added; saturation clamps rather than wraps, and 7 clamping events were seen.
- **Input scaling.** The model expects MIT-BIH-like amplitudes in millivolts within ±8 and one lead (MLII). Another front end or lead would need its own scaling and, in general, retraining.
- **Floor rounding.** The arithmetic shifts round toward minus infinity (a small average bias of half a step per layer). This is part of the validated model and is reproduced by the hardware.

Appendix A. Reference integer model

Core of fixed_point_forward_v2.py, the model the hardware reproduces bit for bit. Comments marked CHANGED/NEW record what differs from the Phase 2.1 version.

```
def quantize_input(x, frac_bits=12):
    return np.clip(np.round(x * (2**frac_bits)), -32768, 32767).astype(np.int64)

def sat16(x):
    # real 16-bit signed saturation, applied at EVERY layer this time, not just the end
    return np.clip(x, -32768, 32767)

overflow_count = {'conv1': 0, 'conv2': 0, 'fc1': 0, 'fc2': 0}

def fixed_point_forward_v2(raw_beat):
    x = quantize_input(raw_beat, frac_bits=12)  # Q4.12, unchanged

    # --- conv1: Q4.12 x Q2.14 -> Q26 acc, bias <<12, rescale >>16 -> Q6.10 ---
    w1 = qw['conv1.weight'].reshape(8, 5).astype(np.int64)
    b1 = qw['conv1.bias'].astype(np.int64)
    out_len1 = 260 - 5 + 1
    conv1_out = np.zeros((8, out_len1), dtype=np.int64)
    for f in range(8):
        for i in range(out_len1):
            acc = np.sum(x[i:i+5] * w1[f])
            acc = acc + (b1[f] << 12)
            acc = acc >> 16                       # CHANGED: was >>12, now targets Q10
            conv1_out[f, i] = acc
    overflow_count['conv1'] += int(((conv1_out > 32767) | (conv1_out < -32768)).sum())
    conv1_out = sat16(conv1_out)                  # NEW: real saturation, not just at the end
    conv1_out = np.maximum(conv1_out, 0)           # ReLU
    pool1_out = conv1_out[:, 0::2][:, :conv1_out.shape[1]//2]
    pool1_out = np.maximum(pool1_out, conv1_out[:, 1::2][:, :pool1_out.shape[1]])

    # --- conv2: Q6.10 x Q1.15 -> Q25 acc, bias <<10, rescale >>15 -> Q6.10 ---
    w2 = qw['conv2.weight'].astype(np.int64)
    b2 = qw['conv2.bias'].astype(np.int64)
    out_len2 = pool1_out.shape[1] - 5 + 1
    conv2_out = np.zeros((16, out_len2), dtype=np.int64)
    for f in range(16):
        for i in range(out_len2):
            acc = np.sum(pool1_out[:, i:i+5] * w2[f])
            acc = acc + (b2[f] << 10)             # CHANGED: was <<14
            acc = acc >> 15                        # same shift as before, now relative to Q25
            conv2_out[f, i] = acc
    overflow_count['conv2'] += int(((conv2_out > 32767) | (conv2_out < -32768)).sum())
    conv2_out = sat16(conv2_out)
    conv2_out = np.maximum(conv2_out, 0)
    pool2_out = conv2_out[:, 0::2][:, :conv2_out.shape[1]//2]
    pool2_out = np.maximum(pool2_out, conv2_out[:, 1::2][:, :pool2_out.shape[1]])

    flat = pool2_out.flatten()  # Q6.10, length 992

    # --- fc1: Q6.10 x Q1.15 -> Q25 acc, bias <<10, rescale >>16 -> Q7.9 ---
    w_fc1 = qw['fc1.weight'].astype(np.int64)
    b_fc1 = qw['fc1.bias'].astype(np.int64)
    fc1_out = (w_fc1 @ flat)
    fc1_out = fc1_out + (b_fc1 << 10)             # CHANGED: was <<14
    fc1_out = fc1_out >> 16                        # CHANGED: was >>15
    overflow_count['fc1'] += int(((fc1_out > 32767) | (fc1_out < -32768)).sum())
    fc1_out = sat16(fc1_out)
    fc1_out = np.maximum(fc1_out, 0)

    # --- fc2: Q7.9 x Q1.15 -> Q24 acc, bias <<9, rescale >>14 -> Q6.10 ---
    w_fc2 = qw['fc2.weight'].astype(np.int64)
    b_fc2 = qw['fc2.bias'].astype(np.int64)
    fc2_out = (w_fc2 @ fc1_out)
    fc2_out = fc2_out + (b_fc2 << 9)              # CHANGED: was <<14
    fc2_out = fc2_out >> 14                        # CHANGED: was >>15
    overflow_count['fc2'] += int(((fc2_out > 32767) | (fc2_out < -32768)).sum())
    fc2_out = sat16(fc2_out)

    return fc2_out
```